PROYECTO 2 · MPI Y CRIPTOGRAFÍA

Búsqueda exhaustiva de llaves DES con Open MPI

Implementación, diagramas y evaluación experimental

Computación Paralela · Octavo semestre
15 de octubre de 2026

Integrante	Carné
Por completar	Por completar
Por completar	Por completar
Por completar	Por completar

Alcance del trabajo

Se conserva el archivo base, se corrigen sus defectos de cifrado y búsqueda, y se implementa una solución con memoria privada por proceso. El texto contiene **350 palabras**. La evaluación comprende **180 mediciones reales**, además de calentamientos y pruebas de corrección.

Las mediciones principales recorren un subconjunto de 2^18 candidatos y utilizan DES real de 56 bits. Se comparan intervalos contiguos, distribución cíclica y búsqueda con trabajo fijo. Los procesos se ejecutan en un Apple M1 Pro; el estudio no incluye una red de varias máquinas.

PROYECTO 2 / COMPUTACIÓN PARALELA

1. Fundamentos de DES

DES es un cifrador **simétrico**: la misma llave secreta permite cifrar y descifrar. Aunque el enunciado usa “llave privada”, aquí no interviene un par de llaves pública y privada. DES transforma bloques de 64 bits y utiliza 56 bits efectivos; los otros ocho bits de su representación se destinan a paridad. [1]

Transformación de un bloque

La permutación IP reorganiza la entrada y la divide en L_0 y R_0, de 32 bits. En cada una de las 16 rondas se calcula L_i = R_(i-1) y R_i = L_(i-1) XOR f(R_(i-1), K_i). Después se concatena R_16 con L_16 y se aplica IP inversa. [1]

La función f expande R de 32 a 48 bits, aplica XOR con la subllave, transforma ocho grupos de seis bits en ocho grupos de cuatro mediante cajas S y permuta los 32 bits resultantes. PC-1 selecciona 56 bits de llave; rotaciones de dos mitades de 28 bits y PC-2 generan las 16 subllaves de 48 bits. Para descifrar se usan en orden inverso. [1]

Preparación del texto en esta implementación

Paso	Operación que realiza el programa
1. Codificar	Leer el archivo UTF-8 como bytes. Las palabras se cuentan aparte; DES trabaja con bytes.
2. Rellenar	Agregar PKCS#7 hasta completar un múltiplo de 8 bytes. Se agrega relleno incluso si la longitud ya es múltiplo de 8. [3]
3. Cifrar	Aplicar DES-ECB a cada bloque completo con la misma llave. ECB no necesita vector de inicialización. [2]
4. Guardar	Escribir bytes binarios. No usar strlen para medir el criptograma.
5. Recuperar	Descifrar todos los bloques, validar y retirar el relleno; buscar la frase exacta.

DES fue retirado como estándar en 2005. En este proyecto se utiliza con finalidad didáctica; ECB también expone la repetición de bloques iguales. [2, 7]

PROYECTO 2 / COMPUTACIÓN PARALELA

2. Diagrama del algoritmo DES

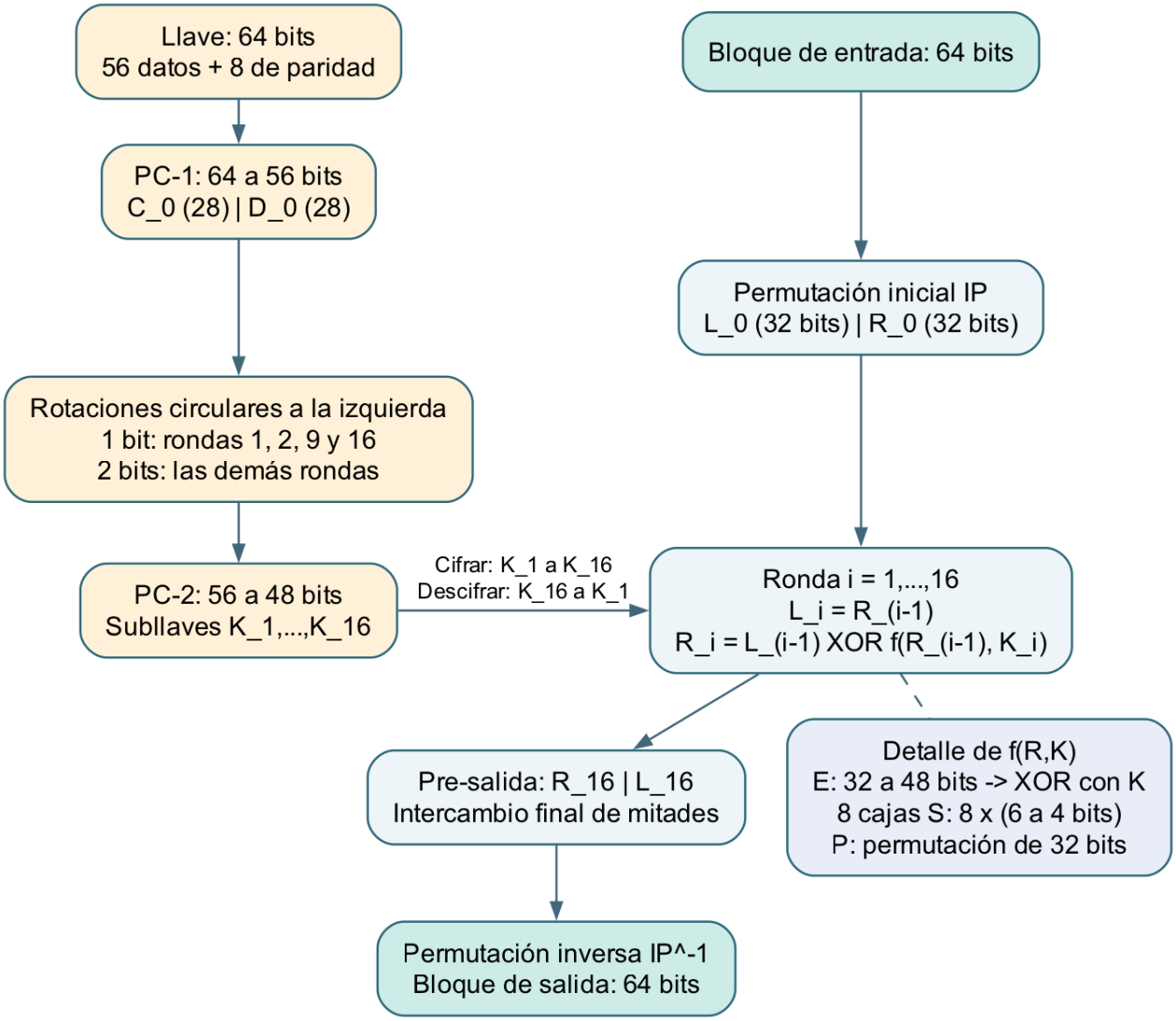


Figura 1. Flujo de un bloque DES y generación de subllaves. Elaboración propia a partir de FIPS 46-3 [1].

Cómo leer el diagrama

La rama derecha transforma el bloque. La izquierda prepara las subllaves que utiliza cada ronda. Para el descifrado se conserva el flujo y se invierte la secuencia de subllaves. El recuadro de f amplía la operación interna de una ronda; no representa una ronda adicional.

Por ejemplo, invertir una ronda permite recuperar R_(i-1) = L_i y L_(i-1) = R_i XOR f(L_i, K_i). Esta identidad explica por qué la estructura de Feistel permite recuperar los datos sin invertir individualmente las cajas S. La biblioteca ejecuta las tablas de permutación y sustitución; nuestro programa distribuye las llaves candidatas.

PROYECTO 2 / COMPUTACIÓN PARALELA

3. Revisión del archivo proporcionado

El archivo original se preserva en **original/bruteforce_.c**. La compilación directa se intentó con mpicc -O3. En este macOS falla porque no existe la cabecera RPC heredada requerida por el ejemplo:

```
original/bruteforce_.c:6:10: fatal error:
'rpc/des_crypt.h' file not found
```

Hallazgo en el original	Efecto y corrección aplicada
ecb_crypt(..., 16, ...)	Cifra y descifra únicamente los primeros dos bloques. Se reemplaza por un ciclo hasta len, sobre todo el mensaje.
encrypt(key, ciph)	La firma original no recibe len; decrypt sí lo recibe pero lo ignora. Ambas funciones corregidas reciben y usan len.
Frase con espacios externos	La frase original no coincide con “(los mismos del proyecto 1),”. Se elige una frase que aparece literalmente en el texto.
0xFE << i*8	La máscara tiene tipo int. Desplazamientos de 32 bits o más causan comportamiento indefinido en este entorno. Se usan uint64_t y bytes explícitos.
Límites de los rangos	Para ranks intermedios myupper = fin - 1 combinado con i < myupper omite un candidato. Se usan cantidades y límites exclusivos.
MPI_Irecv sobre found	El programa escribe found mientras hay una recepción pendiente en ese mismo búfer. Se evita ese acceso mediante una reducción colectiva.
MPI_Wait sin solución	Si no se detecta la frase, rank 0 espera un mensaje que nadie envía. La versión corregida termina al agotar el trabajo.
Sin cronómetro ni validación	Se agregan MPI_Wtime, conteo por proceso, repeticiones y comparación byte por byte del texto recuperado.

La ejecución válida y las mediciones corresponden a la versión corregida. El original sin cambios no produjo un ejecutable en este entorno; por ello no se presenta un tiempo suyo ni se utiliza como línea base.

PROYECTO 2 / COMPUTACIÓN PARALELA

4. encrypt y decrypt: funcionamiento

Las funciones reciben un índice de llave y un búfer que modifican en el mismo lugar. En el original, ambas construyen un long k, ajustan paridad con des_setparity y llaman ecb_crypt. El parámetro DES_ENCRYPT o DES_DECRYPT elige la dirección; el literal 16 restringe el trabajo a 16 bytes.

```
// Firmas reales del archivo proporcionado:
void encrypt(long key, char *ciph);
void decrypt(long key, char *ciph, int len);

// Contrato de la version corregida:
encrypt(uint64_t key, unsigned char *data, size_t len);
decrypt(uint64_t key, unsigned char *data, size_t len);
```

Conversión portable del índice a la llave DES

Se dividen los 56 bits del índice en ocho grupos de siete. Cada grupo ocupa los bits 7 a 1 de un byte; el bit 0 se ajusta para que el byte tenga una cantidad impar de unos. La enumeración big-endian es explícita: no depende del tamaño de long ni del orden de bytes del procesador.

```
for (unsigned i = 0; i < 8; ++i)
    bytes[7-i] = ((key >> (7*i)) & 0x7f) << 1;
DES_set_odd_parity(&bytes);
DES_set_key_unchecked(&bytes, &schedule);
```

Índice decimal	Llave DES de 8 bytes, con paridad
8,192	0101010101018001
135,168	0101010101104001
249,856	01010101011f4001

DES_set_key_unchecked prepara el calendario de subllaves y permite enumerar también llaves débiles. DES_ecb_encrypt procesa un bloque de ocho bytes en la dirección indicada. Estas funciones están obsoletas desde OpenSSL 3.0; se eligieron para adaptar el ejercicio y mantener visible el costo por candidato. [2]

La conversión nueva es una definición explícita del índice; no pretende reproducir la conversión defectuosa del original. Para comparar resultados entre equipos debe conservarse esta misma convención.

PROYECTO 2 / COMPUTACIÓN PARALELA

4.1 Flujo del cifrado y descifrado

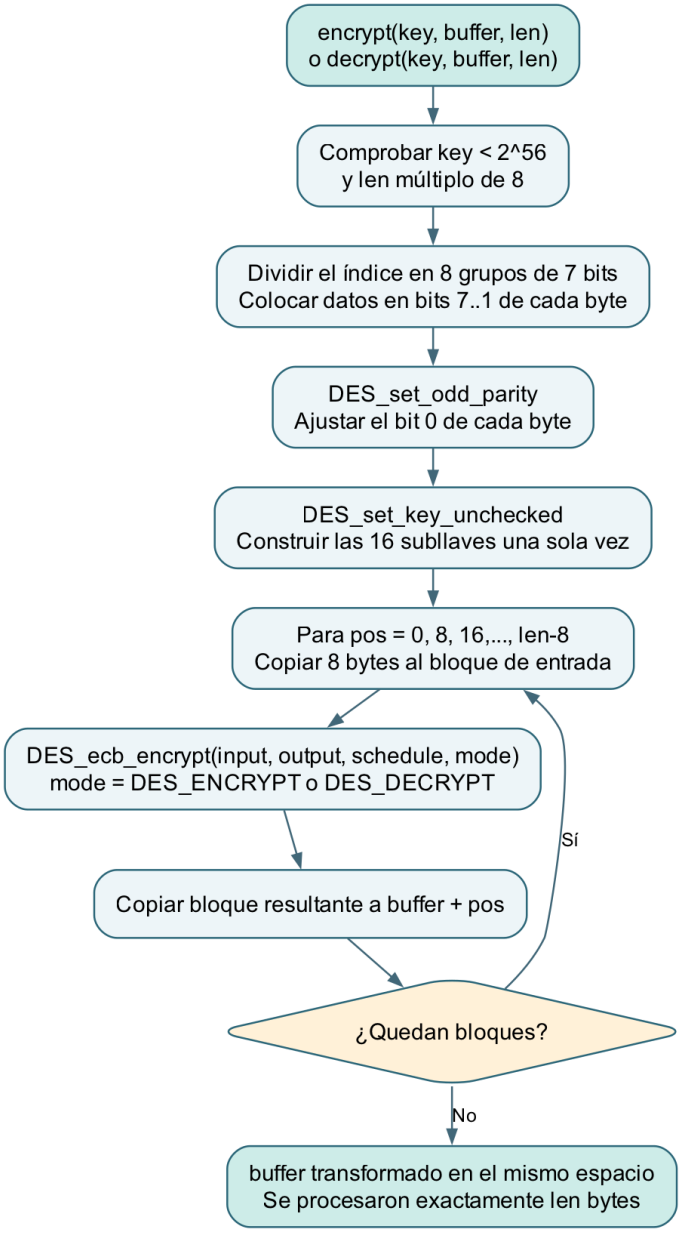


Figura 2. Flujo común de encrypt y decrypt en la versión corregida.

La preparación de subllaves ocurre una vez por llamada, antes del ciclo de bloques. Se usan bloques locales de entrada y salida y memcpy para devolver cada resultado a su posición. El texto de 2 557 bytes recibe tres bytes 03 y produce 2 560 bytes cifrados, distribuidos en 320 bloques.

El descifrado aplica DES a esos mismos 320 bloques; el retiro de PKCS#7 se realiza después. Un resultado binario no se interpreta como cadena hasta colocar un terminador NUL en una posición válida.

PROYECTO 2 / COMPUTACIÓN PARALELA

5. tryKey: comprobar un candidato

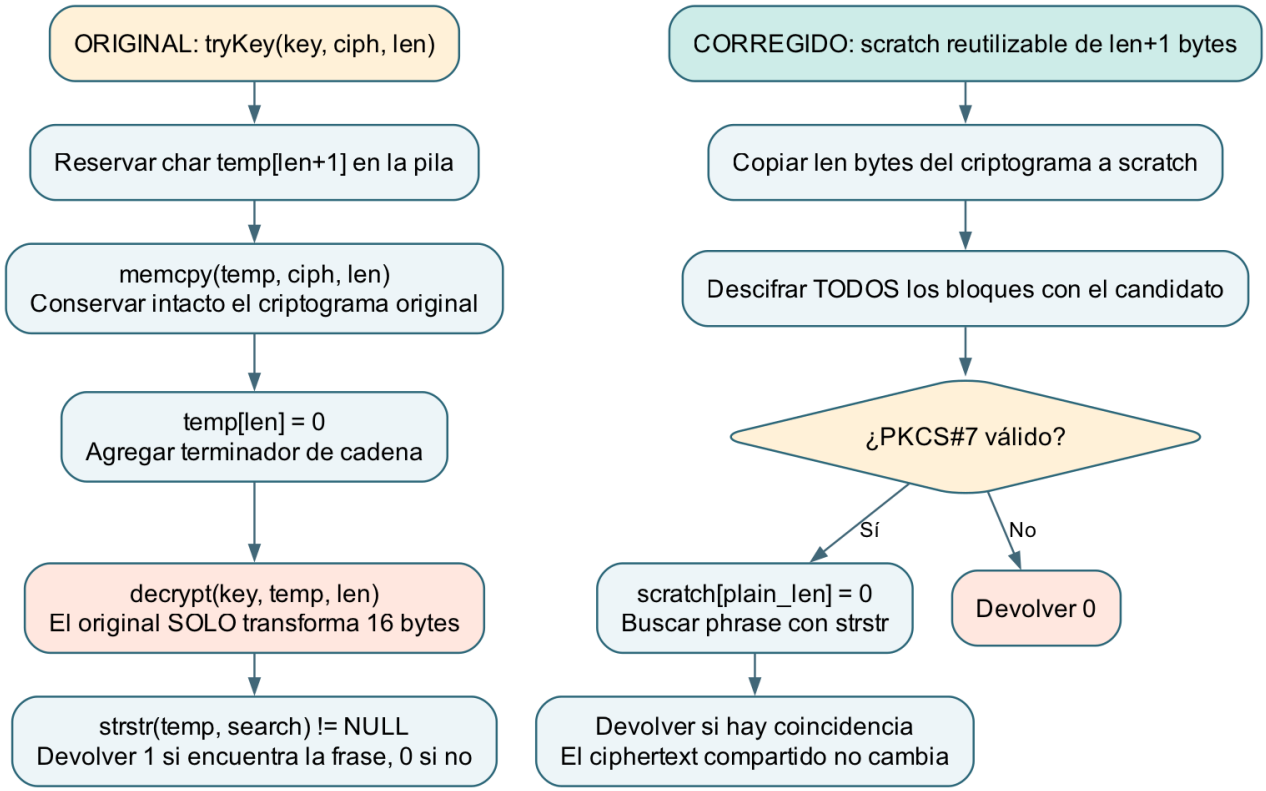


Figura 3. Comparación del flujo original y el flujo corregido de tryKey.

En el original, temp[len+1] es un arreglo local de longitud variable. memcpy copia el criptograma; temp[len]=0 reserva un final de cadena; decrypt modifica la copia y strstr determina si aparece search. El retorno es booleano: 1 cuando el puntero obtenido es distinto de NULL y 0 en otro caso.

En la corrección, scratch se reserva una vez por proceso, evitando crear un arreglo grande en la pila para cada llave. **Cada intento descifra el mensaje completo**. Después se valida el relleno, se fija scratch[plain_len]=0 y se ejecuta strstr. Un relleno inválido descarta el candidato; la frase sigue siendo necesaria para aceptarlo.

La función nunca modifica cipher. Por eso todos los intentos parten de los mismos bytes. Si se descifrara repetidamente el mismo búfer sin restaurarlo, el segundo intento recibiría el resultado del primero y la búsqueda dejaría de ser correcta.

La frase reduce falsos positivos, pero su presencia no constituye una prueba matemática de unicidad. En los experimentos, además se comprobó fuera del tiempo medido que la llave hallada fue la utilizada y que el archivo recuperado coincidió byte por byte con el original.

PROYECTO 2 / COMPUTACIÓN PARALELA

6. memcpy: conservar el criptograma

```
void *memcpy(void *destino, const void *origen, size_t n);
```

memcpy copia n bytes del origen al destino y devuelve el destino. No agrega un terminador ni se detiene ante un byte cero; las regiones no deben superponerse. [4, §7.24.2.1]

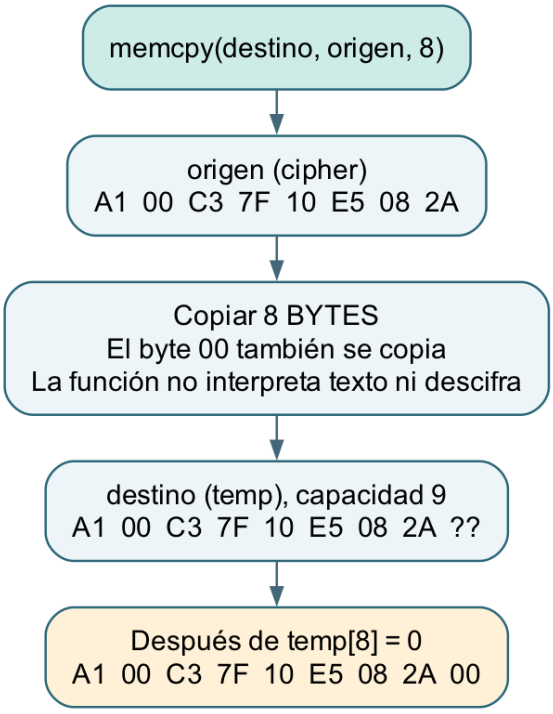


Figura 4. Copia de bytes binarios y terminador agregado por el programa.

Dentro de este programa

La copia inicial permite cifrar un búfer sin perder el texto de referencia. En tryKey se copian exactamente len bytes de cipher a scratch. Cada proceso tiene su propio scratch; MPI no comparte ese puntero entre procesos. El ejemplo de la figura copia el byte 00 de la segunda posición, igual que cualquier otro byte.

El espacio adicional permite escribir el terminador. Ese byte lo escribe el programa, no memcpy. Colocarlo al final evita leer fuera del búfer, pero no elimina posibles ceros internos de un descifrado incorrecto. Por esta razón la longitud del criptograma proviene del archivo, y no de strlen(cipher).

El costo de la copia es lineal en len. En nuestras mediciones se incluye en cada intento, junto con la preparación de subllaves y el descifrado completo; todas las configuraciones ejecutan el mismo trabajo por candidato.

PROYECTO 2 / COMPUTACIÓN PARALELA

7. strstr: validar la frase de búsqueda

```
char *strstr(const char *texto, const char *frase);
```

strstr devuelve un puntero a la primera aparición de la secuencia buscada, o NULL si no existe. La frase vacía devuelve el inicio; por eso el programa la rechaza. Ambas entradas se interpretan como cadenas terminadas en NUL. [4, §7.24.5.7]

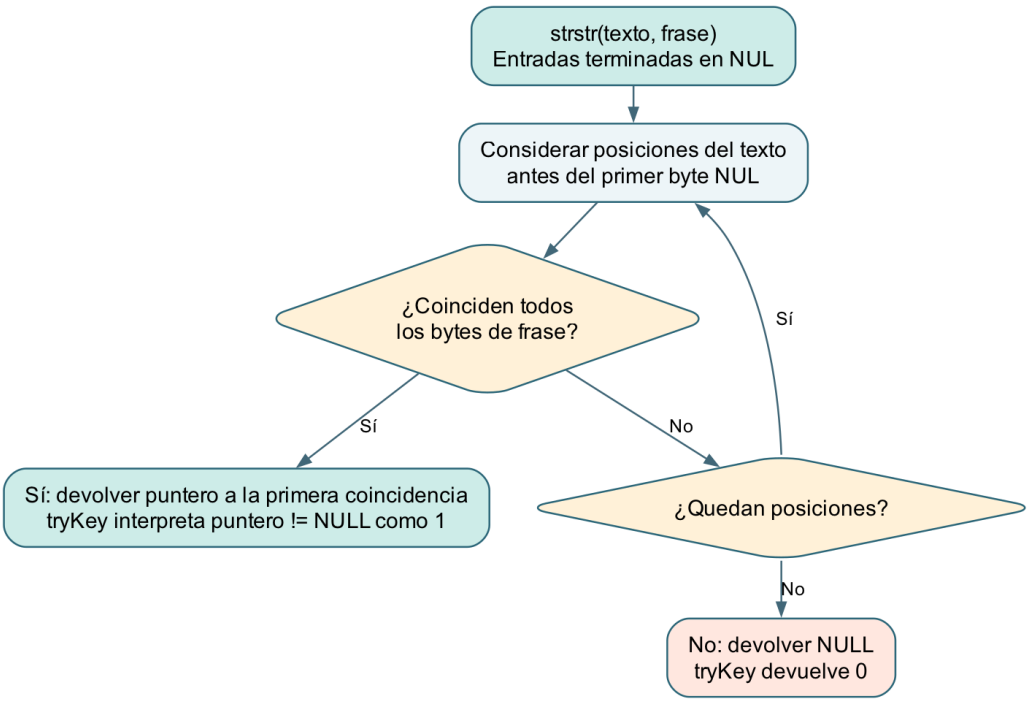


Figura 5. Modelo conceptual de búsqueda; la biblioteca puede usar una implementación optimizada.

Coincidencia literal en el experimento

La frase **ciencia con paciencia** aparece en el segundo párrafo, comienza en el desplazamiento de byte 771 (desde cero) y tiene 21 bytes ASCII. Se compara como secuencia de bytes, incluyendo espacios y mayúsculas. “Ciencia con paciencia” no coincide con nuestra frase. El archivo UTF-8 preserva los acentos del resto del mensaje.

Un descifrado incorrecto puede contener un byte NUL antes de la frase aparente; strstr deja de explorar allí. El mensaje válido no contiene NUL interno. La frase del original, con espacios a ambos lados, no coincide con el fragmento que está entre paréntesis: corregir solo el cifrado no resolvería ese defecto.

Es posible imaginar un buscador binario de longitud explícita para entradas generales. Aquí se mantiene strstr porque el ejercicio utiliza texto y solicita explicar esa función.

PROYECTO 2 / COMPUTACIÓN PARALELA

8. Coordinación de los procesos MPI

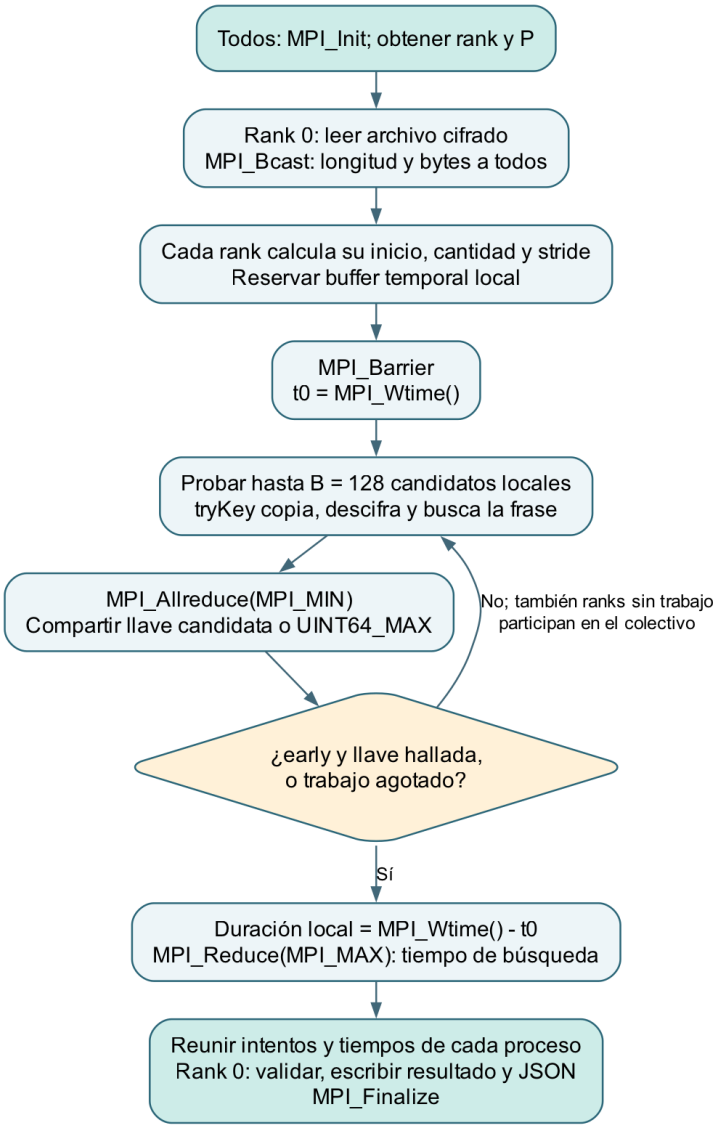


Figura 6. Flujo colectivo de la búsqueda paralela.

MPI_Bcast distribuye el criptograma desde rank 0. Cada proceso conserva una copia y prueba sus candidatos. Cada lote termina con MPI_Allreduce(MPI_MIN): el valor UINT64_MAX representa “sin hallazgo” y cualquier índice válido lo reemplaza. El colectivo entrega el mismo resultado a todos. [5]

Todos ejecutan la misma secuencia de colectivos, incluso los ranks cuyo rango ya se agotó. El modo early termina al detectar una coincidencia global; full continúa hasta cubrir el espacio completo. De esa forma también se resuelve el caso sin coincidencia sin esperar mensajes inexistentes.

El lote de 128 amortiza la coordinación a cambio de trabajo adicional antes de detenerse. En la ronda ganadora, cada rank distinto del ganador puede completar hasta 128 intentos. La memoria total crece con P, mientras que el mensaje de reducción contiene solo un índice de llave.

PROYECTO 2 / COMPUTACIÓN PARALELA

8.1 Distribución del espacio de llaves

El intervalo se define como [L, U), con W=U-L. Para P procesos: q=floor(W/P) y r=W mod P. El rank i recibe c_i=q+1 si i<r y q en otro caso. En la estrategia contigua su inicio es a_i=L+i*q+min(i,r). Cada rank prueba [a_i, a_i+c_i), sin huecos ni duplicación.

En la estrategia cíclica, rank i prueba L+i, L+i+P, L+i+2P, mientras la llave sea menor que U. Su cantidad es cero si i≥W y, en otro caso, 1+floor((W-1-i)/P). La diferencia entre cantidades asignadas no excede una llave.

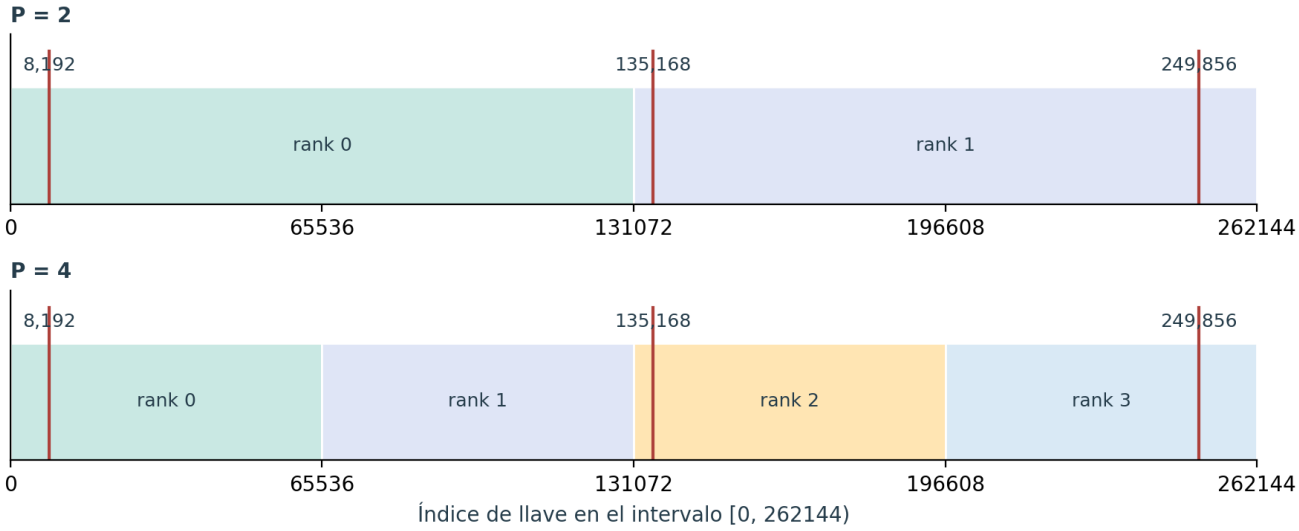


Figura 7. Ubicación de las tres llaves frente a particiones contiguas de 2 y 4 procesos.

Orden de búsqueda y tiempo hasta el hallazgo

Si W es divisible entre P y la llave es K, en intervalos el ganador realiza d=(K-L) mod (W/P)+1 intentos. En cíclica realiza d=floor((K-L)/P)+1. Otros ranks pueden completar el lote de la ronda ganadora. Estas expresiones describen posiciones, no una reducción del costo criptográfico por llave.

Para la llave original K=2^54 y el espacio completo W=2^56, P=4 coloca K exactamente al inicio del rango de rank 1; P=8 la coloca al inicio de rank 2. Una búsqueda secuencial desde cero requeriría K+1 intentos. Este caso muestra por qué una llave elegida favorablemente puede distorsionar el speedup.

PROYECTO 2 / COMPUTACIÓN PARALELA

9. Texto elegido y cifrado completo

350 palabras, contadas por separación en espacios en blanco; **2 557 bytes UTF-8**, incluidos saltos de línea. Frase exacta: **ciencia con paciencia**. El texto se redactó para este ejercicio.

La computación paralela permite resolver problemas mediante la cooperación de varios procesos que trabajan sobre una misma meta. Cada proceso recibe una parte del trabajo y utiliza sus propios recursos para avanzar. Cuando el problema puede dividirse con claridad, esta organización reduce el tiempo necesario para obtener resultados y facilita experimentar con soluciones que serían demasiado lentas en una ejecución secuencial.

En este proyecto estudiamos una búsqueda exhaustiva de llaves para comprender tanto sus posibilidades como sus limitaciones. El mensaje que protegemos describe una experiencia universitaria: un equipo prepara datos, escribe programas, revisa errores y comparte conclusiones sustentadas en evidencia. Nuestra frase de verificación es ciencia con paciencia, porque encontrar una respuesta correcta exige procedimientos cuidadosos y observaciones que puedan repetirse.

El cifrado transforma los bytes del mensaje en bloques que parecen carecer de sentido. Para recuperar el contenido original se necesita aplicar el procedimiento inverso con la llave adecuada. Si esa llave se desconoce, podemos enumerar candidatos, descifrar una copia del mensaje y buscar una frase suficientemente específica. Esta estrategia ilustra por qué el tamaño del espacio de llaves influye directamente en el esfuerzo requerido.

La distribución del trabajo también cambia la interpretación de los tiempos. Una llave cercana al inicio del recorrido puede aparecer pronto, aunque existan muchas combinaciones pendientes. Otra situada al final obliga a realizar más intentos. Además, cuando repartimos intervalos entre procesos, una llave grande puede quedar al comienzo de un intervalo y producir una aceleración aparente especialmente alta. Por eso debemos contar intentos y comparar varias posiciones.

Una evaluación responsable conserva el mismo mensaje, documenta el equipo utilizado y repite cada medición. También distingue el tiempo dedicado al cálculo del empleado en coordinar procesos. Los promedios ayudan a resumir resultados, pero la dispersión revela variaciones que una sola ejecución ocultaría. Finalmente, compartir el código, los comandos y los datos permite que otras personas revisen nuestras decisiones. Así, el aprendizaje combina fundamentos de criptografía, programación distribuida y análisis experimental para explicar resultados con honestidad y precisión. La meta consiste en entender el comportamiento y reconocer qué conclusiones necesitan pruebas antes de generalizarse a distintas redes.

Dato del criptograma principal	Valor
Índice de llave	135168
Llave DES con paridad	0101010101104001
Modo y relleno	DES-ECB y PKCS#7
Tamaño y bloques	2 560 bytes; 320 bloques de 8 bytes
Archivo completo	data/key_135168.des; Base64 íntegro en el Anexo B

PROYECTO 2 / COMPUTACIÓN PARALELA

10. Compilación, ejecución y verificación

El ejecutable corregido se construyó con Open MPI 5.0.11, Apple clang 17.0.0 y OpenSSL 3.6.4. El Makefile detecta la instalación Homebrew de OpenSSL; en Linux permite OPENSSL_PREFIX=/usr. La compilación con -Wall -Wextra -Wpedantic finalizó sin advertencias.

```
make
bin/bruteforce_mpi selftest

bin/bruteforce_mpi encrypt --text data/texto.txt \
  --key 135168 --out data/key_135168.des

mpirun --bind-to none -np 4 bin/bruteforce_mpi search \
  --cipher data/key_135168.des --phrase 'ciencia con paciencia' \
  --upper 262144 --strategy block --mode early \
  --batch 128 --repeats 5 --warmups 1 \
  --out data/recuperado.txt

cmp data/texto.txt data/recuperado.txt
make test
python3 scripts/benchmark.py
python3 scripts/analyze.py
```

Pruebas ejecutadas

Verificación	Resultado
Vector DES de referencia	K=133457799BBCDFF1, P=0123456789ABCDEF, C=85E813540F0AB405. Cifrado y descifrado coinciden.
PKCS#7	34 longitudes, de 0 a 33 bytes, incluidas longitudes múltiplo de ocho.
Límites de búsqueda	Llaves 0, 3, 4, 7, 9 y 10 en [0,11), con 3 procesos y ambas distribuciones.
Casos especiales	Más procesos que llaves; ventanas alrededor de 2^54 y del último índice válido 2^56-1.
Terminación y validación	Ausencia de coincidencia termina con código 2; archivo de longitud inválida se rechaza.
Recuperación completa	22 comprobaciones del verificador, más validación de las 180 corridas medidas. Todos los archivos recuperados coinciden byte por byte.

Los detalles están en results/tests.log y en los archivos de resultados. La búsqueda no recibe --key: la llave se proporciona únicamente al comando encrypt. El verificador externo conoce la llave para comprobar el resultado, después de medir.

PROYECTO 2 / COMPUTACIÓN PARALELA

11. Metodología de las mediciones

Condición	Configuración utilizada
Equipo	Apple M1 Pro; 8 núcleos reportados; 16 GiB de RAM.
Sistema	macOS-26.6.2-arm64-arm-64bit-Mach-O
Entorno paralelo	Un equipo; procesos MPI independientes; --bind-to none; sin fijar afinidad.
Espacio principal	[0, 262144) = 2^18 candidatos. DES conserva las 16 rondas y sus 56 bits efectivos.
Llaves	8192 (3.125% del rango), 135168 (51.5625%) y 249856 (95.3125%).
Configuraciones	3 llaves x 4 cantidades de procesos x 2 distribuciones con parada temprana; más 3 x 4 casos de trabajo fijo con intervalos.
Repeticiones	36 casos; 1 calentamiento descartado y 5 corridas medidas por caso: 180 mediciones y 36 calentamientos.
Orden y lote	Casos en orden aleatorio reproducible, semilla 20261015; repeticiones consecutivas dentro de cada caso; lotes de 128.

Definición exacta del tiempo

Cada corrida inicia con MPI_Barrier. Cada rank mide una duración local con MPI_Wtime y se toma el máximo mediante MPI_Reduce(MPI_MAX). Así no se comparan marcas absolutas de relojes distintos. El intervalo incluye tryKey y los Allreduce; excluye arranque de mpirun, lectura, cifrado, escritura y recolección posterior de estadísticas. [6]

```
media = sum(T_j) / 5
sd = sqrt(sum((T_j - media)^2) / 4)
S_P = media(T_1) / media(T_P)
E_P = 100 * S_P / P
R_P = media(intentos totales) / media(T_P)
```

T_1 es el mismo programa corregido con un proceso, para la misma llave, modo y estrategia. sd es la desviación estándar muestral; las barras de error muestran ±sd. En early, S_P se llama aparente porque puede cambiar el trabajo total. En full, se prueban exactamente 262144 candidatos por corrida y se mide escalamiento con trabajo constante.

No se descartaron valores atípicos de las cinco repeticiones. Las aplicaciones del equipo y el planificador pueden introducir variación; no se afirma aislamiento térmico ni afinidad a un tipo concreto de núcleo.

PROYECTO 2 / COMPUTACIÓN PARALELA

12. Resultados: intervalos contiguos

K	P	Media ± sd (s)	S aparente	Intentos totales
8,192	1	0.1839 ± 0.0055	1.00	8,193
8,192	2	0.1880 ± 0.0018	0.98	16,513
8,192	4	0.1998 ± 0.0123	0.92	33,153
8,192	8	0.4219 ± 0.0424	0.44	66,433
135,168	1	3.0022 ± 0.0297	1.00	135,169
135,168	2	0.0980 ± 0.0047	30.64	8,321
135,168	4	0.1081 ± 0.0168	27.78	16,769
135,168	8	0.2007 ± 0.0074	14.96	33,665
249,856	1	5.5414 ± 0.0523	1.00	249,857
249,856	2	2.6906 ± 0.0097	2.06	237,697
249,856	4	1.2469 ± 0.0051	4.44	213,377
249,856	8	1.0204 ± 0.0804	5.43	164,737

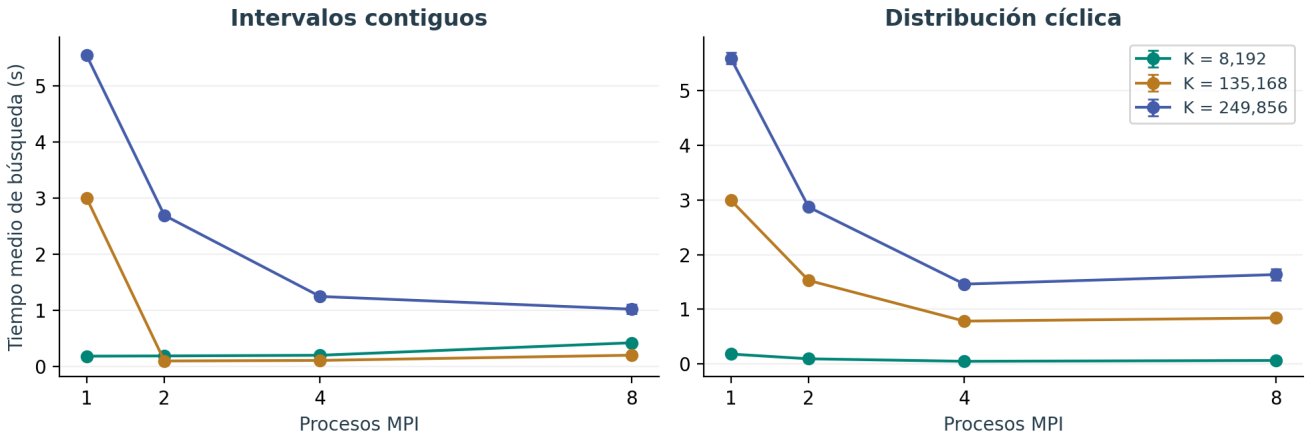


Figura 8. Tiempos con parada temprana; media de cinco corridas y barras de ±sd.

Para K=135168, el tiempo pasa de 3.0022 s a 0.0980 s con dos procesos: S=30.64. La posición local de la llave cae de 135169 intentos a 4097. Ese cambio de trabajo explica el speedup superior al número de procesos.

Para K=8192, el ganador sigue estando en rank 0 y conserva una posición local de 8193 intentos. Agregar procesos apenas acorta ese recorrido y agrega coordinación. El número total de intentos crece porque también trabajan los otros ranks.

PROYECTO 2 / COMPUTACIÓN PARALELA

12.1 Resultados: distribución cíclica

K	P	Media ± sd (s)	S aparente	Intentos totales
8,192	1	0.1831 ± 0.0006	1.00	8,193
8,192	2	0.0968 ± 0.0026	1.89	8,321
8,192	4	0.0510 ± 0.0008	3.59	8,577
8,192	8	0.0660 ± 0.0124	2.78	9,089
135,168	1	2.9934 ± 0.0242	1.00	135,169
135,168	2	1.5272 ± 0.0116	1.96	135,297
135,168	4	0.7851 ± 0.0016	3.81	135,553
135,168	8	0.8433 ± 0.0149	3.55	136,065
249,856	1	5.5857 ± 0.1075	1.00	249,857
249,856	2	2.8670 ± 0.0217	1.95	249,985
249,856	4	1.4607 ± 0.0109	3.82	250,241
249,856	8	1.6347 ± 0.1055	3.42	250,753

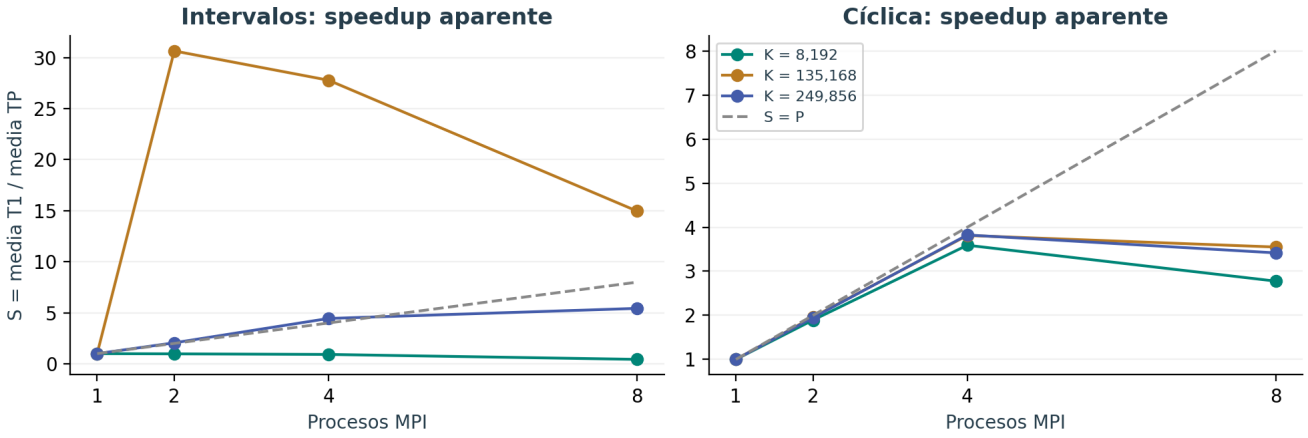


Figura 9. Speedup por llave. La línea discontinua indica S=P; los dos paneles usan escalas verticales distintas.

En cíclica, la posición local depende de floor(K/P)+1. Se mantiene una exploración aproximadamente ascendente del prefijo del espacio y el total de candidatos queda cerca de K+1, más el excedente del lote. Por eso el resultado es menos sensible a que K coincida con una frontera de intervalo.

Con K=8192 y ocho procesos, los lotes cortos y los costos de sincronización son una fracción mayor del tiempo. La estrategia cíclica reduce el sesgo de ubicación, pero no elimina la sobrecarga ni garantiza que ocho procesos siempre sean más rápidos.

PROYECTO 2 / COMPUTACIÓN PARALELA

12.2 Resultados: trabajo fijo

K	P	Media ± sd (s)	S	E (%)
8,192	1	5.7682 ± 0.0190	1.00	100.0
8,192	2	2.9430 ± 0.0137	1.96	98.0
8,192	4	1.5305 ± 0.0075	3.77	94.2
8,192	8	1.7216 ± 0.1956	3.35	41.9
135,168	1	5.7928 ± 0.1033	1.00	100.0
135,168	2	2.9436 ± 0.0108	1.97	98.4
135,168	4	1.6279 ± 0.1800	3.56	89.0
135,168	8	1.6224 ± 0.0624	3.57	44.6
249,856	1	5.7846 ± 0.0148	1.00	100.0
249,856	2	2.9378 ± 0.0060	1.97	98.5
249,856	4	1.5279 ± 0.0029	3.79	94.6
249,856	8	1.5659 ± 0.0293	3.69	46.2

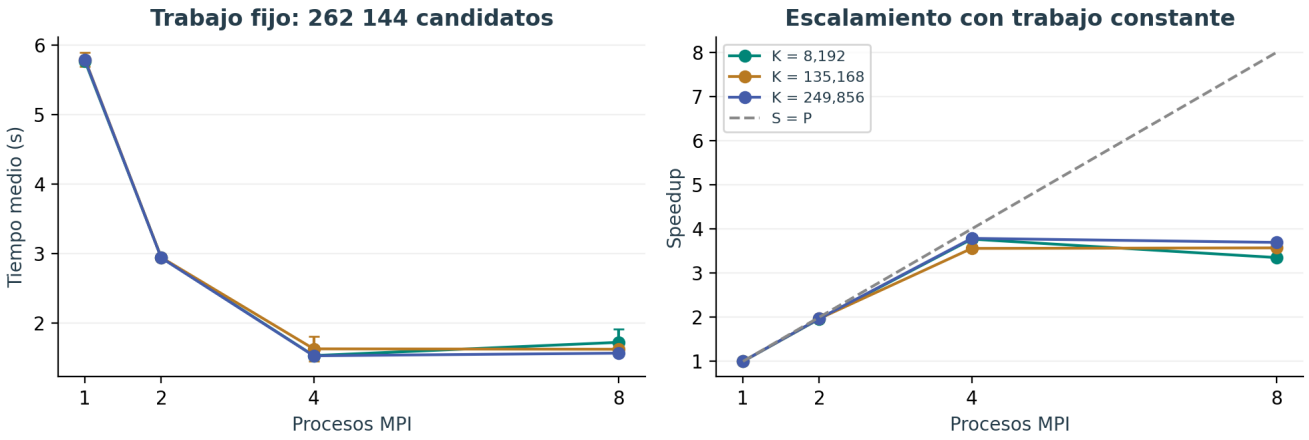


Figura 10. Cada corrida prueba 262144 candidatos aunque encuentre antes la llave.

Con cuatro procesos, el speedup va de 3.56 a 3.79; con ocho, de 3.35 a 3.69. Los cambios de posición de llave ya no alteran el número de intentos. Las diferencias restantes se deben a los tiempos observados de cálculo, espera y ejecución del sistema.

La saturación entre cuatro y ocho procesos muestra el límite práctico en este equipo y con este lote. El experimento por sí solo no permite atribuirla exclusivamente a la red, la caché, la temperatura o un tipo de núcleo.

PROYECTO 2 / COMPUTACIÓN PARALELA

12.3 Trabajo y espera de cada proceso

Rank	Intentos	Cálculo medio (s)	Allreduce medio (s)
0	32,768	0.9490	0.6169
1	32,768	0.9744	0.5914
2	32,768	0.9512	0.6147
3	32,768	0.9484	0.6175
4	32,768	0.9451	0.6207
5	32,768	0.9493	0.6165
6	32,768	0.9518	0.6141
7	32,768	0.9648	0.6010

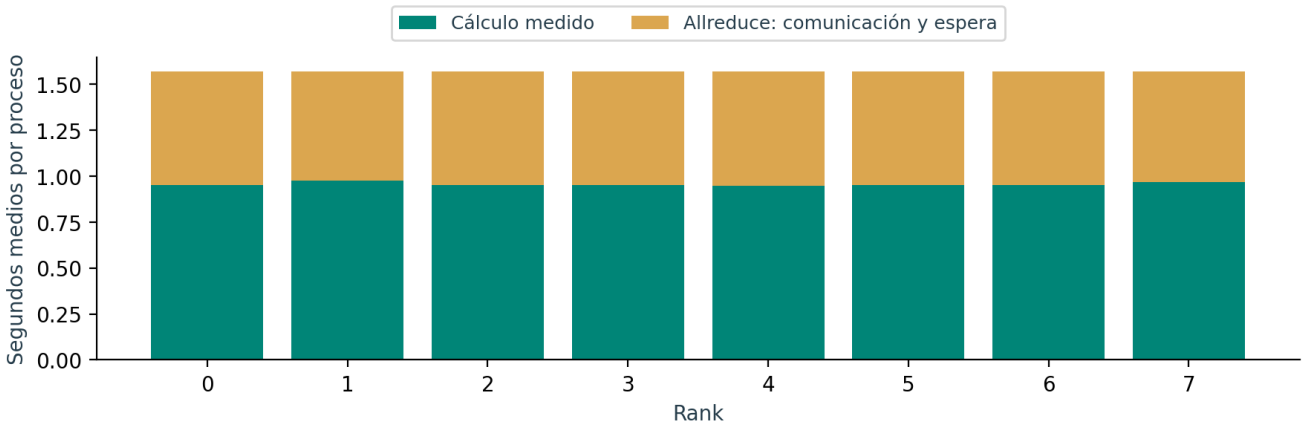


Figura 11. Caso K=249856, P=8, modo full; medias de cinco corridas.

Todos reciben 32768 candidatos. La diferencia de cantidades es cero, pero el tiempo de cálculo varía entre ranks. Los procesos que completan antes un lote pasan más tiempo dentro del colectivo esperando al resto. El tiempo de Allreduce incluye comunicación, espera y posibles pausas del planificador; no es una medición pura de latencia de red.

La suma de cálculo y Allreduce por rank se aproxima a su duración local. Las pequeñas diferencias corresponden al control del ciclo y a llamadas del cronómetro. No se suman las duraciones de todos los procesos para obtener T_P: se usa el máximo.

Consistencia al cambiar la llave

P=1: las medias con las tres llaves abarcan 5.7682 a 5.7928 s; amplitud relativa respecto a su media: 0.42%.

P=4: las medias con las tres llaves abarcan 1.5279 a 1.6279 s; amplitud relativa respecto a su media: 6.40%.

P=8: las medias con las tres llaves abarcan 1.5659 a 1.7216 s; amplitud relativa respecto a su media: 9.52%.

PROYECTO 2 / COMPUTACIÓN PARALELA

13. Por qué unas llaves aparecen antes

La magnitud numérica de una llave no hace más costoso descifrar un bloque DES. El costo cambia principalmente por cuántos candidatos se prueban antes de llegar a ella, por el reparto y por el tiempo hasta comunicar el hallazgo. Con longitud y algoritmo fijos, el trabajo de una prueba sigue siendo esencialmente el mismo.

Estrategia	P	Rank ganador	Posición local de K=135168
block	1	0	135,169
block	2	1	4,097
block	4	2	4,097
block	8	4	4,097
cyclic	1	0	135,169
cyclic	2	0	67,585
cyclic	4	0	33,793
cyclic	8	0	16,897

Interpretación del speedup aparente

En intervalos, K=135168 está 4096 posiciones después del inicio 131072 para P=2, 4 y 8. Todos esos casos tienen la misma profundidad local de 4097 candidatos. El tiempo no se divide nuevamente entre dos al duplicar P; incluso puede aumentar por espera. Una eficiencia aparente superior al 100% refleja una búsqueda diferente, no capacidad de cómputo creada por MPI.

En cambio, K=249856 conserva un recorrido largo dentro de su rango y muestra aceleraciones más moderadas. K=8192 es un caso desfavorable para intervalos porque no se reparte el prefijo que debe recorrer rank 0. Seleccionar únicamente la llave central daría una conclusión sesgada.

Si la llave fuera uniforme en todo el rango

Sin considerar lotes y suponiendo tasas iguales, la profundidad esperada del ganador es aproximadamente W/(2P) tanto con intervalos como con cíclica. Sin embargo, para una misma llave individual, los intervalos pueden presentar saltos bruscos al cambiar P. Tres llaves ilustran este mecanismo; no estiman por sí solas el rendimiento promedio de todas las llaves posibles.

El modo full mantiene el trabajo total y responde a otra pregunta: cuánto acelera MPI una carga fija. Ambos experimentos son útiles, siempre que se informe qué se está comparando.

PROYECTO 2 / COMPUTACIÓN PARALELA

14. Alcance y conclusiones

Lo demostrado con las mediciones

La implementación cifra todo el mensaje y recupera sus bytes originales mediante búsqueda de candidatos. Los rangos tienen cobertura completa, las dos distribuciones terminan colectivamente y el caso sin coincidencia está probado. La búsqueda acepta cualquier índice DES válido y límites hasta 2^56.

La distribución por intervalos produce resultados muy sensibles a la ubicación de K. La cíclica reduce esa sensibilidad en búsquedas desde un límite conocido. El experimento de trabajo fijo permite evaluar el escalamiento sin atribuir a MPI el beneficio de una llave favorable.

Demostración adicional en el espacio de 56 bits

También se cifró el mensaje con la llave del archivo base, K=18014398509481984=2^54, y se buscó con P=4 sobre [0,2^56). En cinco corridas, rank 1 encontró esa llave con media 0.003005 ± 0.000070 s y 385 intentos totales por corrida. El resultado es posible porque la llave ocupa la primera posición de su rango; no representa explorar todo DES.

Límites de lo que se puede concluir

Las 180 mediciones principales utilizan 2^18 candidatos. No se recorrieron exhaustivamente las 2^56 llaves. Los ensayos de llaves grandes prueban direccionamiento y parada temprana en posiciones conocidas; no demuestran un tiempo práctico para romper una llave arbitraria de 56 bits.

La tasa secuencial media en full fue aproximadamente 45,339 candidatos/s. Mantenerla sobre 2^56 candidatos daría unos 50,362 años para el peor caso y la mitad en promedio bajo una llave uniforme. Es una extrapolación lineal orientativa del programa, no un resultado medido ni una estimación de hardware especializado.

Los procesos se ejecutaron en una sola máquina. Para generalizar a memoria distribuida entre máquinas faltaría repetir la matriz con una red real, documentar latencias, arquitectura de cada nodo y política de afinidad, y ampliar el muestreo de llaves. El código emplea tipos MPI de ancho explícito y envía bytes del criptograma para favorecer portabilidad.

PROYECTO 2 / COMPUTACIÓN PARALELA

Referencias y trazabilidad

[1] National Institute of Standards and Technology. (1999). FIPS PUB 46-3: Data Encryption Standard. Especificación de DES y tablas; publicación archivada.
Consultar fuente oficial

[2] OpenSSL Project. DES_random_key y funciones DES. Documentación de OpenSSL 3.6: bloques, paridad, calendario y estado de obsolescencia.
Consultar fuente oficial

[3] Housley, R. (2009). RFC 5652, sección 6.3. Regla de relleno de contenido usada para PKCS#7.
Consultar fuente oficial

[4] ISO/IEC JTC1/SC22/WG14. (2011). Borrador público N1570 de C11, §§7.24.2.1 y 7.24.5.7: memcpy y strstr.
Consultar fuente oficial

[5] Open MPI. MPI_Allreduce. Combinación de valores y distribución del resultado entre procesos.
Consultar fuente oficial

[6] Open MPI. MPI_Wtime. Duraciones locales, unidades y limitaciones de sincronización de relojes.
Consultar fuente oficial

[7] NIST. (2005). Anuncio de retiro de FIPS 46-3. Contexto histórico del uso de DES.
Consultar fuente oficial

Fuentes consultadas el 15 de octubre de 2026. Se usaron fuentes primarias para las especificaciones técnicas. Los diagramas y el análisis de resultados son elaboración propia.

Material y evidencia local

Archivo base entregado: original/bruteforce_.c. Implementación: src/bruteforce_mpi.c. Datos experimentales: results/raw.jsonl; tabla derivada: results/summary.csv; datos por proceso: results/*_ranks.csv; entorno: results/metadata.json. Cada comando exacto se guarda en results/commands.jsonl.

SHA-256 del texto UTF-8:
`4640df7ae35fd17f6eed18ee5bd9dd76858f5f273311a8a15ce84e3d82a0e5ac`

El paquete incluye el código, el texto, los criptogramas, las recuperaciones, los comandos, los diagramas editables y los scripts para volver a producir las tablas y el informe. Los datos medidos corresponden al entorno documentado; una nueva ejecución puede producir tiempos distintos.

PROYECTO 2 / COMPUTACIÓN PARALELA

Anexo A. Las cinco repeticiones de cada caso

Tiempos en segundos, redondeados a cuatro decimales. El promedio y la sd se calcularon con los valores completos de results/raw.jsonl. C=intervalos contiguos; R=cíclica; E=parada temprana; F=trabajo fijo.

K	P / tipo	T1	T2	T3	T4	T5
8192	1 CE	0.1937	0.1814	0.1815	0.1821	0.1806
8192	1 CF	5.7962	5.7750	5.7535	5.7682	5.7482
8192	1 RE	0.1838	0.1836	0.1828	0.1825	0.1830
8192	2 CE	0.1911	0.1875	0.1876	0.1878	0.1861
8192	2 CF	2.9448	2.9367	2.9660	2.9319	2.9358
8192	2 RE	0.1012	0.0970	0.0949	0.0958	0.0949
8192	4 CE	0.1987	0.2213	0.1922	0.1929	0.1937
8192	4 CF	1.5283	1.5394	1.5335	1.5192	1.5323
8192	4 RE	0.0502	0.0503	0.0510	0.0512	0.0523
8192	8 CE	0.4048	0.3881	0.4535	0.3838	0.4793
8192	8 CF	1.5411	1.8957	1.9500	1.6925	1.5289
8192	8 RE	0.0613	0.0530	0.0733	0.0837	0.0585
135168	1 CE	3.0437	3.0235	2.9832	2.9846	2.9762
135168	1 CF	5.7490	5.7771	5.9722	5.7092	5.7563
135168	1 RE	2.9908	2.9852	2.9859	2.9705	3.0345
135168	2 CE	0.0988	0.1060	0.0951	0.0950	0.0949
135168	2 CF	2.9512	2.9576	2.9322	2.9427	2.9344
135168	2 RE	1.5343	1.5297	1.5412	1.5171	1.5137
135168	4 CE	0.1114	0.1362	0.0973	0.0978	0.0978
135168	4 CF	1.5335	1.5207	1.6092	1.9437	1.5325
135168	4 RE	0.7846	0.7836	0.7861	0.7839	0.7874
135168	8 CE	0.1972	0.1919	0.2027	0.2119	0.1997
135168	8 CF	1.6581	1.5799	1.5426	1.7002	1.6313
135168	8 RE	0.8618	0.8407	0.8554	0.8280	0.8305
249856	1 CE	5.5252	5.5255	5.5154	5.6339	5.5070
249856	1 CF	5.8086	5.7879	5.7772	5.7789	5.7702
249856	1 RE	5.5311	5.7705	5.5897	5.5257	5.5115
249856	2 CE	2.6925	2.6980	2.6789	2.7013	2.6826
249856	2 CF	2.9471	2.9403	2.9352	2.9334	2.9329
249856	2 RE	2.9018	2.8592	2.8724	2.8453	2.8565
249856	4 CE	1.2525	1.2394	1.2496	1.2484	1.2444
249856	4 CF	1.5285	1.5266	1.5326	1.5259	1.5259
249856	4 RE	1.4494	1.4566	1.4546	1.4770	1.4657
249856	8 CE	1.0155	0.9514	1.1309	1.0659	0.9380
249856	8 CF	1.6024	1.5341	1.5379	1.5733	1.5819
249856	8 RE	1.4798	1.7741	1.6143	1.6485	1.6568

PROYECTO 2 / COMPUTACIÓN PARALELA

Anexo B. Criptograma completo en Base64

Representación completa de data/key_135168.des, cifrado con índice K=135168. Base64 es solo una codificación para mostrar los bytes; no es el método de cifrado. Al decodificar se obtienen exactamente 2560 bytes.

SHA-256 del criptograma:

```
ac3150377dfb06120f5cecd6230137bab17bd28941b1e8e4e461eaedd5ade675
```

```
t9EpVPbKn2NNdKGisB9DPGZYfbeXF3x/63wLHwio+6eGEq8Fr5+I9CMXegyFPkTuruN27hAnM+ct
2uPLY/WqN7ELxyIwAk9cS5PZKZpBozD9tfsmBbpOFw+8fyW4IZUhPJOAB0k8qUJeFw5QEXNkg1v8
Pu1xWQde1fXJCLcdsiz3P14keSx8/WblT5GawTNeVDkcANtmiWxgSwZcFv/d8uuHZZfK3lclyLrV
eAsmsNTM4j7DdvL+1cs89kCax0vmsv94N3S0jUxmTqYIJPV2nO/XBanTRFEKXYxLqR5yPTaVLl0Z
XRjORnhv6FEcD/bKq8L50FIcTU2d6tCWb2FLGqXfF1YfuFCfcPcFPixQvaZWISUvTbyjgp9ZafZW
NMPSEqFYcL7HhXI7fjlbYNyqlQZAqVWx6VlRmn/6QisjwEpaKyFbRR/h3z1bm+Iv0nRdgIP4mWMm
q/tzx+ZRwZbRSbPKUyOPwq+Xb8bZCShVZHzOGZhPVltr68+Oz8LD20a4wYsSf5i+5c+nJxZL9YlR
5eY/Vle20U9apqtreZ6f0Kn4nvofOa6B2hheWhV23SjOAWLV4zEeasP5xj3weHLNxTx+23OddM7m
d3EQp2b30Vwrd81akrbefJaePAGwGRfR5nuTKbT2dL0a/dyzuelNiFGXYUcguw+/sGOKF882ku3i
/dOKLCMaDovE+xuLGFrWrACbm1c0W2FzfXcZ5EN2HSbm9hsn8sH9AZsdxAhuRJ/AxrTbo7nk3Pny
ZW0hOUFSVD+yslmdlagxXTWFwcqNvus5lyMawM/RdfJ5aRbbGt0RExIlWHrj+wkWajAeLbBxE5q6
oOz5YgLcQcYwyUpLFiDKmsQVqpz780JRE7S1qLOdYuiKJLZlsbhmOx17EJ5ebK4H7o72J5XnhATL
t2IQMrgKP5ukj4dbNJzE+FWRw5wMtKcWfAiaTwW9iIF8I6Xze8j0dDTOR4LsfzrR64J51iuemU7q
8280woDtUTMrCwFCiRls9nWCfj59jylc4Mn1B/wxWvZCCzTPBQKozeyGvoDmo/4xi2CNIXEnQweZ
o7abnB1/Td6PfkoM6kgp4ZABsZSNvQZ3RYaubehOoKx82chyQdcEar3cDV9ZVh/sma2FDKix4SRG
lGfLsfFqSMnC8v0xKar7U2e0TzuLXPj0OXpaCMQosVf4jzQbVhpK1ZsLLvT+bD3U1ytoUELP5c+r
ORXk5B2SMHapfbPyVddaHNMNu0Cf9QKaZJPg1j75X+LmGs/puMBA0hDgXSXX44s085WvIR4ESC6H
WOducoFmQwBviJKUK65PHFlVqKmf958usgVx7mfeJkl3NH33ONA8lI8YpLd4yoPmSYo94aPZka+d
z6mUBk7W9bt8x2AWhpiWAohKBm6wvqfWCAvDu71tu3MHdMe+7RXbmN6PYznlrXYCmI9tZmcn4rsl
Lsd2yeFlOda8lQ6+jKQSgv781jMNjXOaoG7RcACAx3t6mGOg+F6Q8VkFiYBlnQEigd8RMXcpZywo
82hQMK9VhqVToig85u1CXY4Uo89QIqAmYPS9hZVssa2J1IucR1WGrSPho041z/TDXMFt2etxi0Dd
agmptJXE/fJCtzBopG9BXcmajhmH+OvT+KFiakbC9byMYO3quV5Lh2CrqQctDfMUm09NKl2276iI
G2h7K7OJfy7Sdd4IQiGfyzkBPBW0S9RGwqLpQuS827dJRRr93LO56U2ImGmF9NaG1qPcsVM6GR0l
/SM6k+Cz0z0UFLbLSBto44/JcFk+XkuIwA9Y5caeH0quK62dJPtQOUDtsao3Aspd3KjgwKo2AKKK
xZUAn8f/F3KBbrhXjUSVvq8og81wnKor883wMmgd6VOqlneIg0Biy3gMMrZxVUO6WbpuUZw9tUis
3CUblETojzLNQa+eCR9vwYYzmOk0JJnetAzA5EMo3C7TepiNT0o+nvfBqoOztB/ftcjpCHblTdPB
W1DDdMl7z9XRmsrHfT9VSbt7cgueXvZX1tGgF3Kqf7Kq0KIMYpI2UtjYWempr4ZXPMOqR8poT8c4
cPL0qmCfIxzpzhd1eBt/SaHzNhAWhgPZPnkWFGJ+4rAolOkfQy/yrhx7q9lb9HdZE7FD4d23UETW
bDQbSnKNuM8S0jzP+YZ8YfJQCk6npybFZuiNr/RL3PnWZO55XqtBQhC4EFEZX/dJp60WGcBQu1Ks
MDpQGm/XvO8F57jQ99epCQg1ys0X/70AGWJrLtAc8VoVQ7+Fxp2A80b05jtG4k/KpTiKEnMvYgYu
htcAca5fEKNE20QFPW68LR17I28CfAjBA+bK2JbchAVThd77fXu57O3CGrXdfNwvAABstKTnw0fl
aIIl0ALpO0oV870pd04Tgq/3dhAyWWQ+yl4V9SCc/Ej9Qv1jUmzwUylkw/sYheC41vVezcuGiRAf
YvvQURPcEfc4emLc98cD5L3peQXgF0jCJCDQNHJPyXBF0Ka0lz359RR8MGcZdd/uR1WGrSPho06+
+FUkCRdiszJ2llnmsh1SdR8nE7z4LHHDwuRfCG19NFGUPqFdOFuLMwDQIJU5uh8rjGWlRZYO3cxf
fAM5UJseKvt4y0LgSenehblJ5Dhgggh//Mz8F4heYmFhuHyyevjR4PaAjXmIDRNc0XaOMuWZTPKF
EKLpBsfFb4eOO/MmLrwvtE788Zv0V1WZUbdPn4oHp1f7ZWYDZZehYlMglhGKrYTPbRCCZRSO4vD+
vKODBz1bm+Iv0nRdLBI7Em3knfooSKwNAZ4v1wKPyPNL0cm6wK4K4Fcbj57RLCp5S9q5EowZ7WCa
zXV+phjBIpq9kiKEWpizamRpeIj9vDmzRJXqyJZHW8X4EogKlJ/NKPE+DbgKP5ukj4db0TzNvNFB
9cFQS0yN/HENN5R72aN8PueEe1MqpfyRhWxr9uN9xVEzdnH9+PPDzMJClQbY16qcNDPsKFp9GY9j
i8BcHjqn8FKYOndbpVUcflTqVvT6a6J5y003cYAVlori9jW48DNvnBsKpp4Vh6IJAOxyHGuR+0jy
+6e2GsMKOTXRY0fQ9gXomK3Wv5N9nZlULqxpBuB6ntvssHtIm1rvbXsNvbp1OrYOQNbqs9/hbE0N
8b+EEf5eZt+eU9D22c0hOnPrUyqgMlx8Uv8Kt7YPSRJHrjPywtLdfX8cKmwfQTJS9qy90+cSJfZu
PzZ5IsU3+5TEoUhMr2k2K0pPc+TbethSD+P6aoe64L9V6r+PpQZQHfYXSLS62eyDwRz3xlKY1nrL
zwc5kR41byc3UJXiEzRarT8tnFM272I7Dv7gXepPGK2TrpABXEwNZUhlcKMw9XyJtMm73W2vPXK7
76YCB2ptuSzbLQe4nhMrHC5yAVYYtF/GUcvvHbdFNpQiOJNJRzhcTS8seMywk+UmqHmvqA==
```

El archivo binario está incluido en el paquete. Para comprobar el descifrado, ejecutar el comando search de la sección 10 y comparar el archivo recuperado con data/texto.txt. El ciphertext no almacena la llave ni el texto plano.